

APIx: Real-time Airfare Price Index for India

SIH 2026 | PS 26056 | MoSPI (DIID) |

1. The pitch

APIx is presented as a **statistical instrument**, not a scraper. Most teams will show a scraper, a database and a line chart. This entry wins on three things:

- **Correct index methodology** that a statistics office would recognise.
- **Compliance-first scraping**, which the PS explicitly requires.
- **A nowcast** that predicts DGCA monthly average fares before DGCA publishes them.

2. Architecture

SOURCES	INGESTION	LAKE / STORE	INDEX ENGINE	SERVE
Airline sites	+ Scheduler (Prefect/cron)	Raw snapshots	Cleaning + fare split	FastAPI (OpenAPI,
OTAs	+> Compliance Firewall ----->	(Parquet + HTML/JSON	Quality/fare-class norm	SDMX-style JSON)
Fallback APIs	(robots/ToS registry,	evidence, immutable)	Jevons elementary index	Dashboard (React/Next
DGCA/eSankhyiki	+ rate limiter, audit log,	-> PostgreSQL/Timescale	Lowe/Laspeyres upper lvl	+ ECharts)
	kill switch)	-> DuckDB analytics	Lead-time elasticity	NL "Ask APIx"
	Playwright/Scrapy workers		Nowcast vs DGCA	Data-lineage viewer
	Adapter per source		Anomaly + event flags	

Inside the pipeline, raw quotes pass through a **Normalizer** before they reach the index engine (see sections 5 and 6).

3. Layer by layer

3.1 Compliance Firewall (differentiator)

- Source registry stores each site's robots.txt verdict, ToS notes, allowed crawl rate and permitted/blocked status.
- Every request passes a token-bucket rate limiter with jitter and is written to an audit log.
- Global kill switch disables any source instantly.
- **Do not build CAPTCHA-solving.** The PS mentions CAPTCHAs, but bypassing them contradicts its own ToS-compliance requirement. Design: detect CAPTCHA, back off, mark the quote missing, fall back to another source. State this on a slide.
- Load is tiny: 25 routes x 2 directions x 5 lead times = about 250 searches per source per day.

3.2 Collection design (sampling frame)

- **Basket:** top 25-30 city pairs from DGCA traffic data, weighted by passenger share.
- **Lead times:** T+1, T+7, T+15, T+30, T+45 for each route.
- **Time bands:** for example morning, midday, evening.
- **Fields:** carrier, flight number, fare class, cabin and check-in baggage, refundability, base fare, YQ fuel surcharge, UDF, PSF, GST, convenience fee, total, sold-out flag.

3.3 Cleaning pipeline

- Deduplicate quotes across OTAs and airlines by (flight, date, fare class).
- Split base fare, taxes and fees. Compute a **consumer-paid price** and a **pure-fare price**.
- Remove outliers with a robust MAD or IQR filter per route and lead time.
- Treat sold-out flights as censored observations, not zeros.
- Assign each quote a confidence score from cross-source agreement and snapshot age.

3.4 Index construction

- **Elementary level:** Jevons (geometric mean) index per route x lead-time cell, the standard elementary aggregate in CPI practice.

- **Upper level:** Lowe/Laspeyres-type index weighting routes by DGCA traffic, then lead times by a booking-window mix.
- **Method switcher:** toggle Jevons, Dutot and GEKS in the dashboard with a sensitivity panel showing how much the choice matters.
- **Frequencies:** daily as the main output; weekly and monthly as roll-ups. Rebase to 100 at a chosen base date and chain.

3.5 The crazy features

- **Nowcast:** a small model (gradient boosting or dynamic factor) predicts DGCA's monthly average fare from APIx before release; show nowcast vs actual in the back-test.
- **Lead-time elasticity curves:** price ratio as departure approaches, by route type, festival weeks vs normal weeks.
- **Event and anomaly annotator:** auto-flag spikes (festivals, disruptions, fare caps) and annotate the chart.
- **CPI bridge:** APIx's implied contribution to Transport and Communication inflation, using a citable weight structure.
- **Lineage viewer:** click any index point, drill down to route, then quote, then stored raw evidence.
- **Ask APIx:** natural-language box that calls your API (for example, why did DEL-BOM spike last week?).
- **Machine-consumable API:** OpenAPI spec, API keys and SDMX-style JSON so NSO and RBI can pull it directly.

4. Two data channels: airline direct and aggregators

Airfare data reaches APIx through two different kinds of source. They are scraped separately, tagged separately, and reconciled against each other.

	Channel A: Airline direct	Channel B: Aggregators (OTAs)
Sources	IndiGo, Air India, Air India Express, Akasa Air, SpiceJet	MakeMyTrip, Yatra, EaseMyTrip, Cleartrip, Ixigo, Goibibo
Coverage	One carrier per site; the fare-family list is complete (Saver, Flex, etc.)	Many carriers in one search; good for coverage and cross-checks
Price shown	Fare with a breakup (base, taxes and fees) available at the fare summary step	Often a headline total that may or may not include taxes; convenience fee and coupons appear late in checkout
Baggage	Set by fare family; rules are stated on the site	Shown inconsistently; some listings include check-in baggage, some do not
Role in index	Reference price for the index	Validation, gap-filling, and a separate OTA spread series (OTA price minus airline price for the same flight)

- **Reconciliation:** when the same flight, date and fare class appears on both channels, match them and store both quotes. The gap is a finding in itself (convenience fees, discounts, stale OTA cache).
- **Why airline direct is the reference:** it is the closest thing to a source of truth and has the cleanest fare-family and baggage rules. This is a methodology choice; it can be revisited if MoSPI prefers the lowest consumer-payable price across channels.
- **Provenance:** every quote carries a channel tag (DIRECT or OTA) and a source id, so any index point can be traced back to its channel.

5. Normalization layer: tax basis and baggage

Sources disagree on what a price means: some show fare and tax separately, some show a tax-inclusive total, and some bundle baggage. No quote enters the index until it has been converted to one canonical definition. The normalizer sits between the raw quote store and the standardized price DB.

5.1 Step 1: detect the price basis of every quote

- Each source adapter declares a `price_basis` label: **BASE_ONLY**, **BASE_PLUS_TAX_SEPARATE** or **ALL_INCLUSIVE**.
- Do not trust the label alone. Wherever possible the scraper opens the **fare breakup** step and captures the components. Verify each site's behaviour by hand once, then keep it as a regression test.
- If only a total is visible, estimate taxes from a tax-rules table (airport charges by airport, GST by cabin class, fuel surcharge) and set `taxes_estimated = true`. Estimated rows get a lower confidence score.

5.2 Step 2: decompose into canonical components

Every quote is rewritten as: **total = base fare + fuel surcharge + airport charges (UDF, PSF) + GST + other taxes + convenience fee - discount**.

- Reconciliation check: the sum of components must match the displayed total within a small tolerance. If not, the row is flagged and excluded from the index until reviewed.
- Two prices are derived: **fare_ex_tax** (base plus fuel surcharge) and **total_consumer_paid** (what the traveller actually pays).

5.3 Step 3: normalize baggage

- Capture `bag_cabin_kg` and `bag_checkin_kg` for every quote, and derive a `bag_tier`: **CABIN_ONLY**, **CHECKIN_15**, **CHECKIN_20PLUS**.
- Choose one **reference product** for the index (for example Economy, cabin baggage only). Quotes in other tiers are either (a) adjusted by the observed price of the add-on baggage, or (b) kept in their own stratum and never mixed with the reference tier.
- This is a quality adjustment: the index must track the price of the same product over time, not a product that quietly changed from no-bag to bag-included.
- Output is `total_ref_baggage`, the total price restated for the reference baggage tier.

5.4 What the normalizer does for each case

Source shows	Normalizer action
Base fare with taxes listed separately	Capture components; total = sum; check against displayed total
Tax-inclusive total only	Back out taxes with the tax-rules table; mark <code>taxes_estimated</code> ; lower confidence
Total includes check-in baggage	Tag <code>bag_tier</code> ; subtract observed add-on baggage price or move to a separate stratum
OTA adds convenience fee or coupon at checkout	Store as separate components; index uses the fee-free airline price, OTA spread series uses the fee-inclusive one
Price cannot be reconciled	Flag, exclude from index, keep raw evidence for review

6. Standardized Price DB (schema)

Two tables. `raw_quotes` is immutable and holds exactly what the site showed. `std_prices` holds the normalized rows the index reads. The ten core fields below are the ones in your design; the extras make normalization and auditing possible.

Field	Type	Notes
route	text	Origin-destination pair, for example DEL-BOM
airline	text	Operating carrier
travel_date	date	Date of departure
booking_datetime	timestamp	When the quote was observed; <code>lead_days</code> is derived from it
fare_class	text	Fare family, for example Saver or Flex
base_fare	numeric	Canonical base fare in Rs
taxes	numeric	Total taxes and fees, broken down in the extra columns

total_fare	numeric	total_consumer_paid after normalization
baggage	text	Baggage entitlement, stored as bag_tier plus kg fields
source	text	Site name; paired with channel (DIRECT or OTA)
Extra columns		flight_no, dep_time, lead_days, channel, price_basis_raw, fuel_surcharge, airport_charges, gst, convenience_fee, discount, taxes_estimated, bag_cabin_kg, bag_checkin_kg, total_ref_baggage, refundable, sold_out, confidence, raw_evidence_uri, scrape_run_id, normalization_version

Data flow: scraper adapters write to raw_quotes, the normalizer (price basis, components, baggage) writes std_prices, and the index engine reads only from std_prices.

7. The back-test problem, honestly

The PS demands 30 days of results against DGCA data, but 30 days of real scrapes cannot exist by tonight. Recommended approach:

- Start real scheduled scraping now so live data accumulates.
- Backfill the back-test using DGCA's published data plus a **clearly labeled synthetic generator** calibrated to real fare behaviour (lead-time curve, weekday effect, festival lift).
- Mark synthetic data as synthetic in the UI and in the deck. Overclaiming is the one thing that can sink you with an informed judge.

8. Tech stack

Layer	Choice
Scraping	Playwright (Python), per-source adapters, stored HTML/JSON evidence
Storage	Postgres, or DuckDB + Parquet if short on time
Index engine	Python (pandas or polars) with pytest tests
API	FastAPI
Frontend	React/Next with ECharts, or Streamlit for a fast demo
Scheduling	cron or Prefect
Packaging	Docker Compose

9. What to build today, in priority order

1. **Standardized price DB and normalizer** (tax basis and baggage), tested on a few real quotes from each channel.
2. **Index engine and API on seeded data.** Core of the score.
3. **Dashboard:** APIx line, route heatmap, elasticity curve, back-test chart vs DGCA.
4. **One or two real scraper adapters** (one airline site, one OTA) to prove it works.
5. **Compliance firewall stub** with a visibly working rate limiter, robots check and audit log.
6. **The PPT.**
7. **The nowcast**, only if time remains; otherwise present it as a designed module.

10. PPT skeleton (about 12 slides)

1. Problem and why manual CPI collection misses online fares
2. Solution overview and the APIx name
3. Architecture diagram
4. Ethical scraping and compliance firewall

5. Two-channel collection: airline direct plus aggregators
6. Normalization: tax basis and baggage
7. Standardized price DB and cleaning pipeline
8. Index methodology (Jevons plus weighted aggregation)
9. Dashboard screenshots and API
10. Back-test results and nowcast
11. Innovation, scalability and impact for MoSPI/RBI
12. Roadmap and team